ArtNet-ETH

4-fach Art-Net/DMX/RDM-Node mit echtem Ethernet auf Basis des **WIZnet W5100S-EVB-Pico** (RP2040 + W5100S + RJ45).

Das Projekt ist eine Abwandlung von `ArtNet-USB`: **eine Firmware** enthält sowohl den onboard-W5100S über SPI (RJ45) als auch **Ethernet über USB (CDC-NCM)**. Beim Booten prüft der Node, ob ein USB-Datenhost das Composite-Gerät enumeriert:

- **USB-Datenhost vorhanden** → Netzwerk über USB (CDC-NCM), RJ45 bleibt ungenutzt
- **nur USB-Stromversorgung** (Ladegerät) oder kein USB → W5100S + RJ45

Die Transportwahl ist damit eine Laufzeit-Entscheidung; es gibt keine separate USB-Firmware mehr. Beide Transporte nutzen dieselben DMX/RDM-, Web- und OLED-Quellen.

Funktionsumfang

- 4 DMX-Ports, jeder einzeln als **Ausgang** (ArtDmx → DMX out) oder **Eingang** (DMX in → ArtDmx) konfigurierbar
- vollständiges **Art-Net 4**: ArtPoll/ArtPollReply, ArtDmx, ArtAddress
- **RDM-Gateway** (ArtRdm, OpCode `0x00CC`) auf den Ausgangsports
- **Web-Oberfläche** mit Kanalübersicht und Einstellungen
- **lokales OLED-Menü** mit 4 Tasten (Status, Ports, DHCP, Neustart, Werkseinstellungen)
- **DHCP-Client mit statischem Fallback** im RJ45-Modus (Standard: `2.0.0.10/8`)
- **automatische Transportwahl** beim Booten: USB-Datenhost → **CDC-NCM** („Ethernet über USB“) mit **DHCP-Server für den Host**, sonst W5100S + RJ45
- kontinuierliche DMX-Ausgabe mit ~33 Hz, auch wenn der Controller nur Änderungen sendet

Hardware

Bauteil	Funktion
U5	WIZnet W5100S-EVB-Pico (RP2040, W5100S, RJ45)
U11–U14	MAX485 (im Schaltbild als MAX3485EPA bezeichnet), RS485-Transceiver je DMX-Port
U1–U4	CT3-12AFAP-EP, 3-polige XLR-Einbaubuchse
U100–U103	120 Ω, Terminierung je Port
TERM1–TERM4	DP-1-R-T, Schalter für die 120-Ω-Terminierung
U113–U120	560 Ω Bus-Bias (A → 5 V, B → GND)
R40–R43	10 kΩ in Reihe zwischen RO und RP2040
D1–D4	SM712-02HTG, Überspannungsschutz A/B gegen GND
OLED	SSD1306 128×64 an I2C1
KEYS	4 Tasten gegen GND

Stand des Schaltbilds: **V1.0, 2026-11-08**. Die Transceiver werden im Schaltbild aus **5 V (VBUS)** versorgt, das OLED aus 3,3 V.

Die DMX-Ports sind als 3-polige XLR-Buchsen ausgeführt. Das ist im DMX-Umfeld nicht standardkonform (Norm: 5-polig XLR), aber verbreitet. Belegung: **1 = GND/Schirm, 2 = Data−, 3 = Data+**. Kabel entsprechend beschriften, damit sie nicht mit symmetrischen Audiokabeln verwechselt werden.

Alle vier Buchsen sind **weiblich** (`CT3-12AFAP-EP`), also das normgerechte Geschlecht für DMX-**Ausgänge**. Ein Port lässt sich zwar per Firmware auf Eingang umschalten, DMX-Eingänge sind laut Norm aber **männlich**. Für den Eingangsbetrieb ist daher ein männlicher Geschlechtsadapter bzw. ein Kabel mit zwei Steckern nötig. Wer die Ports überwiegend als Eingang nutzt, sollte die Buchsen gegen die Male-Variante tauschen (z. B. `CT3-05M-EP`) und für den Ausgang einen Adapter verwenden.

Pin-Belegung

Funktion	GPIO
DMX TX / DI (Treiber-Eingang)	GP2, GP3, GP4, GP5
DMX RX / RO (Empfänger-Ausgang)	GP10, GP11, GP12, GP13
DE/RE (Richtung, HIGH = senden)	GP6, GP7, GP8, GP9
OLED SDA	GP26 (I2C1, OLED-Pin 4)
OLED SCL	GP27 (I2C1, OLED-Pin 3)
Taste UP	GP14 (`KEYS`-Pin 3)
Taste DOWN	GP15 (`KEYS`-Pin 4)
Taste OK	GP22 (`KEYS`-Pin 2)
Taste BACK	GP28 (`KEYS`-Pin 1)
Status-LED	GP25 (onboard)
W5100S SPI0 MISO / CS / SCK / MOSI	GP16 / GP17 / GP18 / GP19
W5100S RST / INT	GP20 / GP21

`KEYS` ist ein 1×6-Stiftleiste: **1 = GP28, 2 = GP22, 3 = GP14, 4 = GP15, 5 = frei, 6 = GND**. Die Firmware geht von dieser Reihenfolge aus (BACK, OK, UP, DOWN). Ist das Tastenfeld anders herum bestückt, nur die `KEY_*`-Konstanten in `src/board.h` tauschen.

`OLED` ist ein 1×4-Stiftleiste: **1 = 3,3 V, 2 = GND, 3 = SCL/GP27, 4 = SDA/GP26**.

> **Wichtig:** GP16–GP21 sind auf dem W5100S-EVB-Pico fest mit dem onboard-W5100S verbunden und dürfen **nicht** für Tasten oder andere Funktionen verwendet werden. Im finalen Schaltbild sind diese Pins am Trägerboard bewusst unbeschaltet; der `KEYS`-Header liegt auf GP14/15/22/28.

Adressierung der DMX-Ports

Jeder Port hat eine eigene Art-Net-Adresse aus **Net** (0–127), **Subnet** (0–15) und **Universe** (0–15). Werkseinstellung: Net 0, Subnet 0, Universe 0–3.

Netzwerk

RJ45-Modus (W5100S)

Der W5100S wird über `Wiznet5100lwIP` (lwIP, nicht blockierend) betrieben. Beim Start:

1. Ist DHCP aktiviert, wird bis zu **8 s** auf ein Lease gewartet.
2. Kommt kein Lease, wird die konfigurierte statische Adresse verwendet (Standard `2.0.0.10`, Maske `255.0.0.0`).
3. Ohne DHCP (`DHCP client` aus) wird sofort die statische Adresse verwendet.

Die Web-Oberfläche ist danach unter `http://<aktuelle-IP>/` erreichbar. Die aktuell verwendete IP und Maske stehen in der Kopfzeile; im Formular wird die **statische Fallback-Adresse** bearbeitet.

USB-Modus (CDC-NCM)

Wird beim Booten ein USB-Datenhost erkannt, ist das USB-Kabel die Netzwerkschnittstelle. Der Node nutzt immer seine konfigurierte **statische Adresse** (Standard `2.0.0.10/8`); einen DHCP-Client gibt es hier nicht. Stattdessen kann der Node einen **DHCP-Server für den Host** betreiben:

- `DHCP` **an** (Standard): Der Host bekommt automatisch eine Adresse aus demselben Subnetz, Pool `.1`–`.9` der Node-IP (also z. B. `2.0.0.1`–`2.0.0.9`).
- `DHCP` **aus**: Der Host muss manuell im Node-Subnetz konfiguriert werden, z. B. `2.0.0.2/8`.

Moderne Windows-10/11-, macOS- und Linux-Systeme erkennen CDC-NCM ohne Treiberinstallation als Netzwerkadapter. Die Web-Oberfläche ist unter `http://<Node-IP>/` erreichbar, im NCM-Modus also standardmäßig unter `http://2.0.0.10/`.

Das Gerät meldet sich am USB als **„VID-PRO ArtNet Node“** mit Hersteller `VID-PRO` an (gesetzt in `initVariant()` über `USB.setProduct()` / `USB.setManufacturer()`).

> **Windows 10/11 – „Code 10“ nach KB5124008:** Das Windows-Update vom September 2026 verschärft die Prüfung der NCM-NTB-Parameter in `usbncm.sys`. TinyUSB liefert standardmäßig `wNdbInDivisor` und `wNdbOutDivisor` mit Wert `1`; Windows quittiert das mit „Code 10 – schwerwiegender Hardwarefehler“. Deshalb liegt in `lib/tusb-ncm` eine lokale, gepatchte Kopie des NCM-Class-Treibers, die beide Divisoren auf `4` setzt (wie TinyUSB-PR #3914). Diese lokale Bibliothek überschreibt automatisch die gleichnamige Core-Bibliothek.
>
> Unter Windows 10 kann für den NCM-Adapter zusätzlich einmalig der Treiber „Microsoft → UsbNcm Host Device“ gebunden werden; Windows 11 erkennt CDC-NCM normalerweise von selbst.

Der NCM-Modus verwendet eine feste MAC `02:00:00:00:00:01`, die zum USB-Deskriptor der eingebundenen `lwIP_USB_NCM`-Bibliothek passt. Die NCM-Schnittstelle wird in `initVariant()` registriert, damit sie bereits bei der ersten USB-Enumeration im Deskriptor steht (wichtig für macOS). Genau diese Enumeration dient auch der Erkennung: Ein reines Ladegerät meldet das Gerät nie als „mounted“, ein Datenhost schon.

In beiden Modi läuft Art-Net auf UDP-Port **6454**. Broadcasts verwenden die Maske der aktiven Schnittstelle, nicht die gespeicherte Fallback-Maske.

> Wenn das Netzwerk beim Booten nicht bereit ist, versucht die Firmware den Aufbau aus `loop()` erneut. Ein nachträglich eingestecktes Kabel führt bei aktivem DHCP im RJ45-Modus allerdings nicht automatisch zu einem neuen Lease – in diesem Fall neu starten oder statisch konfigurieren. Im NCM-Modus wartet der Node nach einem Warmstart darauf, dass der Host die Schnittstelle neu enumeriert.
>
> Der W5100-Treiber der Arduino-Pico-Bibliothek meldet keinen PHY-Linkstatus. Die Anzeige „link up“ im OLED bedeutet daher, dass der Netzwerk-Stack bereit ist, nicht zwingend, dass ein Kabel steckt. Im NCM-Modus bedeutet sie, dass der Host die USB-Netzwerkschnittstelle enumeriert hat.

Transport-Auswahl im Quellcode

`src/main.cpp` enthält beide Netzwerk-Objekte (`Wiznet5100lwIP` und `NCMEthernetlwIP`). In `setup()` wartet `detectTransport()` bis zu `board::USB_DETECT_TIMEOUT_MS` (Standard **2000 ms**) auf `tud_mounted()`:

- **mounted** → `usb_ncm = true`, NCM-Bring-up mit fester NCM-MAC, statischer Node-Adresse und optionalem DHCP-Server
- **nicht mounted** → `usb_ncm = false`, W5100S-Bring-up mit DHCP-Client und statischem Fallback

`initVariant()` registriert die NCM-Schnittstelle immer, damit der Datenhost die erste Enumeration bereits als CDC + NCM sieht, und setzt die USB-Namen (`VID-PRO` / `VID-PRO ArtNet Node`). Bei reiner Stromversorgung wird die Schnittstelle nicht aktiviert.

Die beiden Transporte sind bewusst **gegenseitig ausschließend**: Es gibt keinen Dual-Netif-Betrieb. Das vermeidet mehrdeutiges Art-Net-Broadcast- und Reply-Routing, wenn RJ45 und USB gleichzeitig aktiv wären. Eine Umschaltung zur Laufzeit (Kabel nach dem Booten) ist nicht implementiert; die Wahl steht nach dem Booten fest. Die NCM-Bibliothek liegt eingebunden unter `lib/lwIP_USB_NCM`; der Windows-kompatible NCM-Class- Treiber liegt als lokaler Override unter `lib/tusb-ncm`.

Web-Oberfläche

- **Channels**: 512 Kanäle des gewählten Ports, Suche und „Active only“
- **Settings**:
  - IP und Netzmaske (RJ45: statischer Fallback; NCM: Node-Adresse)
  - DHCP ein/aus (RJ45: Client; NCM: Server für den Host)
  - ShortName/LongName für ArtPollReply (Standard: `VID-PRO ArtNet` / `VID-PRO 4x ArtNet Node`)
  - pro Port: Richtung (output/input), Net, Subnet, Universe
  - Speichern + Neustart, nur Neustart, Werkseinstellungen

OLED-Menü und Tasten

Taste	Funktion
UP / DOWN	Statusseite wechseln, Menüeintrag wählen, Wert ändern
OK	Menü öffnen, Eintrag auswählen, nächstes Feld
BACK	Menü verlassen, Port-Editor beenden und speichern

Statusseiten: IP/Netzwerkstatus, Portübersicht mit Aktivitätsanzeige, Systeminformationen (Laufzeit, FPS, DHCP-Status). Im NCM-Modus zeigt die Statusseite „USB link up“ und im Systemmenü den DHCP-Server-

Status.

Menüpunkte: Port 1–4 bearbeiten, DHCP ein/aus (RJ45: Client, NCM: Server für den Host), Speichern + Neustart, Werkseinstellungen, Zurück. Eine Änderung der DHCP-Einstellung im Menü wird gespeichert und ist beim nächsten Neustart aktiv.

DMX/RDM-Technik

- **DMX-Ausgabe**: eigener PIO/DMA-Transmitter (`DmxOut`) auf `pio0`, ein State Machine pro Port, Break/MAB und 250 kbaud nach DMX512.
- **DMX-Eingang**: PIO-UART-Empfänger auf `pio1`, ein State Machine pro Port. Die empfangenen Frames werden als ArtDmx gesendet.
- **RDM**: `RdmPort` schaltet DE/RE für die Anfrage auf Senden und horcht danach auf die Antwort. ArtRdm wird nur auf Ausgangsports beantwortet.
- Alle 8 PIO-State-Machines werden benötigt (4× TX auf `pio0`, 4× RX auf `pio1`).

Bauen und Flashen

Voraussetzung: PlatformIO mit dem Arduino-Pico-Core (`earlephilhower`). Das Board ist auf `wiznet_5100s_evb_pico` eingestellt. Eine Firmware enthält beide Transporte; die Auswahl passiert beim Booten automatisch.

```
# Bauen, flashen, Konsole (eine Firmware für RJ45 und USB-NCM)
pio run -d ArtNet-ETH
pio run -d ArtNet-ETH -t upload
pio device monitor -d ArtNet-ETH

# oder aus dem Repo-Root
pio run -e artneteth
make build-artneteth
```

Alternativ die `.uf2` im BOOTSEL-Modus auf das Board kopieren:

Firmware	Projekt-Build	Root-Env
RJ45 + USB-NCM	`.pio/build/w5100s/firmware.uf2`	`.pio/build/artneteth/firmware.uf2`

Die serielle Konsole läuft über native USB mit 115200 Baud. Im NCM-Modus teilen sich Konsole (CDC) und Netzwerk (NCM) denselben USB-Port; die Konsole zeigt unter anderem den erkannten Transport, DHCP-Status, verwendete IP, OLED-Erkennung und Port-Richtungen an.

Hinweise zum Schaltbild

Die folgenden Punkte sind beim Abgleich des EasyEDA-Projekts (`hardware/ProPrj_miniQ_2026-11-08.epro2`) mit der Firmware aufgefallen:

1. **MAX485 an 5 V**: Im Schaltbild ist der Transceiver als **MAX3485EPA** beschriftet; tatsächlich bestückt ist der **MAX485** (5-V-Typ, pin- und funktionskompatibel). Deshalb ist VCC (Pin 8) an **5 V (VBUS)** richtig. Die Beschriftung/BOM im EasyEDA-Projekt sollte auf MAX485 korrigiert werden. Bei 5 V liefert RO

ebenfalls ~5 V; die 10-kΩ-Reihenwiderstände R40–R43 begrenzen nur den Strom in die Eingangsklemmen des RP2040 und ersetzen keinen echten Pegelwandler.
2. **Bias**: 560 Ω von A nach **+5 V**, 560 Ω von B nach GND. Der Bias ist fest bestückt und nicht abschaltbar. Bei einer Umstellung auf 3,3 V muss er mit auf 3,3 V. Bias nur einmal pro Bus vorsehen.
3. **Terminierung**: U100–U103 (120 Ω) liegen in Reihe mit TERM1–TERM4 zwischen A und B. Die 120-Ω-Terminierung nur an den beiden physischen Busenden einschalten – nicht zusätzlich an jedem Knoten, sonst sinkt der Buswiderstand (z. B. 60 Ω bei zwei eingeschalteten Nodes).
4. **XLR3**: siehe oben – 3-polige XLR ist keine normgerechte DMX-Verbindung. Pin 2 = Data− (B), Pin 3 = Data+ (A). Alle Buchsen sind weiblich; für den Eingangsbetrieb ist ein male-male-Adapter erforderlich (siehe Hardware-Abschnitt).
5. **KEYS/OLED**: Die Header-Belegung steht in der Pin-Tabelle. GP16–GP21 sind am Trägerboard absichtlich frei, da sie intern zum W5100S gehen.
6. **TVS/SM712**: A und B über SM712 gegen GND geschützt. Schirm von GND trennen bzw. nach Norm behandeln.
7. **5-V-Versorgung**: Der 5-V-Rail kommt vom VBUS-Pin des W5100S-EVB-Pico, also nur bei USB-Versorgung. Wird ausschließlich über VSYS gespeist, sind die Transceiver unversorgt; dann 5 V extern zuführen.

Die Schaltbild-Beschriftung (MAX3485EPA) und der Bestückungstext sollten auf den tatsächlich verwendeten **MAX485** vereinheitlicht werden; die 5-V-Versorgung und der Pegel am RP2040 sind dann wie oben beschrieben zu behandeln.

Unterschiede zu ArtNet-USB

	ArtNet-USB	**ArtNet-ETH**
Netzwerk	CDC-NCM über USB	automatisch: W5100S + RJ45 oder CDC-NCM (USB-Datenhost)
DHCP	DHCP-Server für den Host	RJ45: DHCP-Client mit Fallback; NCM: DHCP-Server für den Host
USB	Netzwerk + Konsole	RJ45: nur Konsole/Flashen; NCM: Netzwerk + Konsole
Tasten	keine	4 Tasten + OLED-Menü
DMX/RDM	4 Ports	4 Ports, gleiche PIO-Aufteilung

Die DMX/RDM-Bibliotheken (`DmxOut`, `ArtnetRdm`, `ArtnetConfig`) und der SSD1306-Treiber sind aus den bestehenden Projekten übernommen.